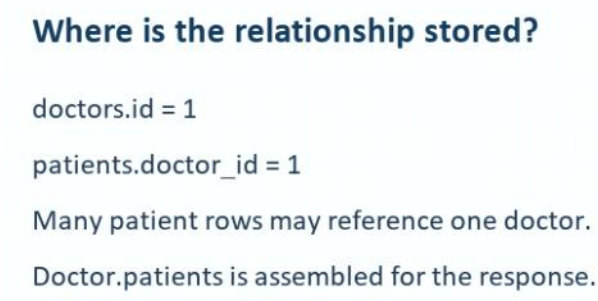
Lecture 11 - Hospital practice and Java revision

Wednesday, October 28, 2026 6:09 PM

Lecture 11 — Hospital practice and Java revision

0. Homework check

Which database column stores a patient's doctor assignment? Where is a doctor's patient list assembled? What should happen when the same assignment is repeated?



1. Doctor–Patient association: requirements

One doctor can have many patients. A patient can be unassigned or assigned to one doctor at a time. The existing H2 database stores doctors and assignments.

PUT /doctors/{doctorId}/patients/{patientId} assigns an existing patient and returns the doctor with their current patients. GET /doctors/{doctorId}/patients returns that doctor's patient list. An existing doctor without patients returns 200 with []. Missing doctor or patient IDs return 404 with a useful message.

Repeating the same assignment must not duplicate patients. Assigning a different doctor moves the patient. Failed assignments leave existing data unchanged. Assignments survive restart, and results reflect renamed or deleted patients.

2. H2 schema and sample data

Use the existing H2 file database and working directory. Retain completed homework schema; the following statements create only missing structures. A non-null patients.doctor_id must reference an existing doctors.id. NULL means no doctor is assigned.

```
CREATE TABLE IF NOT EXISTS doctors (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

ALTER TABLE patients
    ADD COLUMN IF NOT EXISTS doctor_id BIGINT;

ALTER TABLE patients
    ADD CONSTRAINT IF NOT EXISTS fk_patients_doctor
    FOREIGN KEY (doctor_id) REFERENCES doctors(id);
```

Check existing rows before inserting sample doctors. Run this INSERT only when both sample IDs are absent; insert only the missing row if one already exists.

```
SELECT id, name FROM doctors ORDER BY id;

INSERT INTO doctors (id, name)
VALUES (1, 'Dr Asha'), (2, 'Dr Meera');
```

3. Doctor response model and missing-doctor error

Doctor.patients is a response list assembled from current database rows. It does not persist the association by itself. The association is stored in patients.doctor_id.

Doctor.java

```
package com.accio.springProject.Hospital.Model;

import com.accio.springProject.Hospital.Entity.Patient;
import java.util.ArrayList;
import java.util.List;

public class Doctor {
    private final long id;
    private final String name;
    private List<Patient> patients = new ArrayList<>();

    public Doctor(long id, String name) {
        this.id = id;
        this.name = name;
    }

    public long getId() { return id; }
    public String getName() { return name; }
    public List<Patient> getPatients() { return patients; }
    public void setPatients(List<Patient> patients) {
        this.patients = patients;
    }
}
```

DoctorNotFoundException.java. Reuse the existing PatientNotFoundException and ApiError.

```
package com.accio.springProject.Hospital.Exception;

public class DoctorNotFoundException extends RuntimeException {
    public DoctorNotFoundException(long id) {
        super("Doctor " + id + " not found");
    }
}
```

4. Repository: SQL and named row mappers

findById reads a doctor. assignPatient updates the patient's doctor_id. findPatientsByDoctorId reads current patient rows. RowMapper converts one result row into a Java object.

```
package com.accio.springProject.Hospital.Repository;

import com.accio.springProject.Hospital.Model.Doctor;
import com.accio.springProject.Hospital.Entity.Patient;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

@Repository
public class DoctorRepository {
    private final JdbcTemplate jdbc;
```

```
    private final RowMapper<Doctor> doctorMapper = (rs, rowNum) ->
            new Doctor(rs.getLong("id"), rs.getString("name"));

    private final RowMapper<Patient> patientMapper = (rs, rowNum) ->
            new Patient(rs.getLong("id"), rs.getString("name"), rs.getInt("age"));

    public DoctorRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

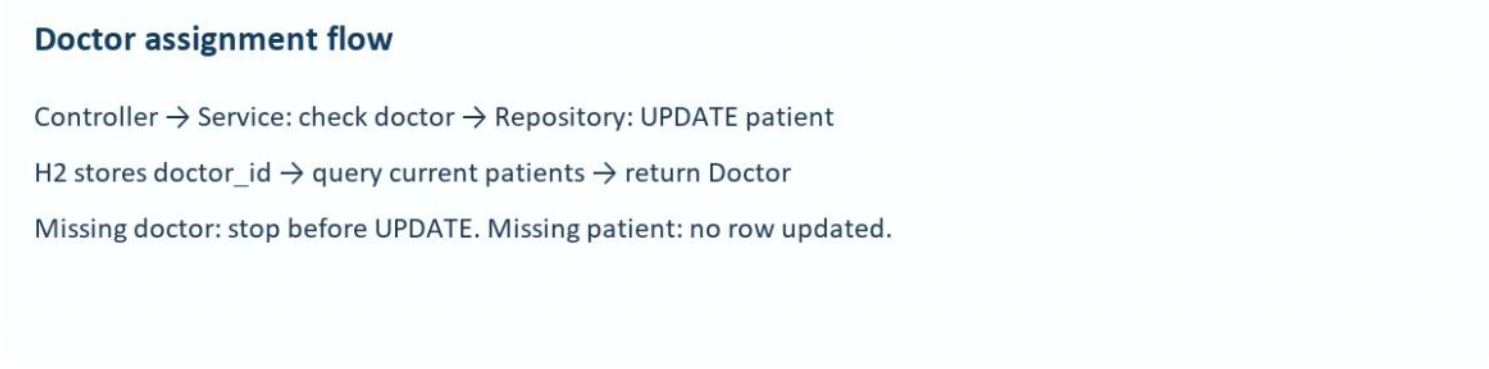
    public Doctor findById(long id) {
        List<Doctor> matches = jdbc.query(
                "SELECT id, name FROM doctors WHERE id = ?",
                doctorMapper,
                id);
        return matches.isEmpty() ? null : matches.get(0);
    }

    public boolean assignPatient(long doctorId, long patientId) {
        int changedRows = jdbc.update(
                "UPDATE patients SET doctor_id = ? WHERE id = ?",
                doctorId, patientId);
        return changedRows > 0;
    }

    public List<Patient> findPatientsByDoctorId(long doctorId) {
        return jdbc.query(
                "SELECT id, name, age FROM patients WHERE doctor_id = ? ORDER BY id",
                patientMapper,
                doctorId);
    }
}
```

5. Service: assignment rules

Validate the doctor before updating a patient. A missing doctor must not replace an existing assignment.
Read the current patient list after a successful update.




```
package com.accio.springProject.Hospital.Service;

import com.accio.springProject.Hospital.Model.Doctor;
import com.accio.springProject.Hospital.Entity.Patient;
import com.accio.springProject.Hospital.Repository.DoctorRepository;
import com.accio.springProject.Hospital.Exception.DoctorNotFoundException;
import com.accio.springProject.Hospital.Exception.PatientNotFoundException;
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class DoctorService {
    private final DoctorRepository repository;

    public DoctorService(DoctorRepository repository) {
        this.repository = repository;
    }

    public Doctor assign(long doctorId, long patientId) {
        Doctor doctor = requireDoctor(doctorId);
        if (!repository.assignPatient(doctorId, patientId)) {
            throw new PatientNotFoundException(patientId);
```

```
        }
        doctor.setPatients(repository.findPatientsByDoctorId(doctorId));
        return doctor;
    }

    public List<Patient> getPatients(long doctorId) {
        requireDoctor(doctorId);
        return repository.findPatientsByDoctorId(doctorId);
    }

    private Doctor requireDoctor(long id) {
        Doctor doctor = repository.findById(id);
        if (doctor == null) {
            throw new DoctorNotFoundException(id);
        }
        return doctor;
    }
}
```

6. Controller: HTTP responses

The controller reads path variables, calls the service and translates missing-resource exceptions to 404 responses. The repository handles SQL; the service handles the operation's rules.

```
package com.accio.springProject.Hospital.Controller;

import com.accio.springProject.Hospital.Model.ApiError;
import com.accio.springProject.Hospital.Service.DoctorService;
import com.accio.springProject.Hospital.Exception.DoctorNotFoundException;
import com.accio.springProject.Hospital.Exception.PatientNotFoundException;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/doctors")
public class DoctorController {
    private final DoctorService service;

    public DoctorController(DoctorService service) {
        this.service = service;
    }

    @PutMapping("/{doctorId}/patients/{patientId}")
    public ResponseEntity<Object> assign(
            @PathVariable("doctorId") long doctorId,
            @PathVariable("patientId") long patientId) {
        try {
            return ResponseEntity.ok(service.assign(doctorId, patientId));
        } catch (DoctorNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        } catch (PatientNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        }
    }

    @GetMapping("/{doctorId}/patients")
    public ResponseEntity<Object> getPatients(@PathVariable("doctorId") long doctorId
) {
        try {
            return ResponseEntity.ok(service.getPatients(doctorId));
        } catch (DoctorNotFoundException exception) {
            return ResponseEntity.status(404)
                    .body(new ApiError(exception.getMessage()));
        }
    }
}
```

7. Request evidence

Verify each behaviour using actual returned patient IDs. Retain the same H2 file URL and working directory when checking restart persistence.

```
GET /doctors/1/patients
→ 200 [] if none are assigned

POST /hospital/addPatient
{"name":"Meera","age":26}
→ use the returned patient's actual ID below

PUT /doctors/1/patients/{patientId}
→ 200, Doctor with its current patient list

Repeat the same PUT
→ same assignment; patient appears once

GET /doctors/1/patients
→ list containing the assigned patient

PUT /hospital/updatePatient/{patientId}
{"name":"Meera Shah","age":26}
GET /doctors/1/patients
→ current name Meera Shah

Stop and restart using the same H2 file URL and working directory.
GET /doctors/1/patients
→ assignment remains

PUT /doctors/999999/patients/{patientId}
→ 404, doctor missing; old assignment remains

PUT /doctors/1/patients/999999
→ 404, patient missing

PUT /doctors/2/patients/{patientId}
→ reassigned to doctor 2
GET /doctors/1/patients → patient is absent
GET /doctors/2/patients → patient is present

DELETE /hospital/deletePatients/{patientId}
GET /doctors/2/patients
→ deleted patient is absent



SELECT id, name FROM doctors ORDER BY id;
SELECT id, name, age, doctor_id FROM patients ORDER BY id;
```

8. Explain the behaviour

Why does an existing doctor with no patients return []? Why does repeating PUT not duplicate the patient? Why does a renamed patient appear with the new name? Why does the Doctor model's list not replace database storage? Which step protects data when the doctor ID is missing?

9. Exercise: patient age-range API

Implement GET /hospital/patients/age-range?minAge=18&maxAge=40. Include both boundaries. Both ages must be between 0 and 120; minAge must not exceed maxAge. Invalid ranges return 400 with an ApiError message. A valid range without matches returns 200 with [].

Use repository.findAll(), a loop and a separate result List. Keep the existing schema and SQL. The endpoint must not modify stored patients.

Where should range validation live? Is an empty result an error? Which boundary operators include ages 18 and 40?



Age-range reference solution: service

```
import com.accio.springProject.Hospital.Entity.Patient;
import com.accio.springProject.Hospital.Exception.InvalidPatientException;
import java.util.ArrayList;
import java.util.List;



public List<Patient> getPatientsByAgeRange(int minAge, int maxAge) {
    if (minAge < 0 || minAge > 120 || maxAge < 0 || maxAge > 120) {
        throw new InvalidPatientException("Ages must be between 0 and 120");
    }
    if (minAge > maxAge) {
        throw new InvalidPatientException("minAge must not exceed maxAge");
    }

    List<Patient> matches = new ArrayList<>();
    for (Patient patient : repository.findAll()) {
        int age = patient.getAge();
        if (age >= minAge && age <= maxAge) {
            matches.add(patient);
        }
    }
    return matches;
}
```

Age-range reference solution: controller

Add the method inside the existing /hospital controller. Missing or non-numeric request parameters may be rejected by Spring before this method body; its catch handles the service's InvalidPatientException.

```
import com.accio.springProject.Hospital.Model.ApiError;
import com.accio.springProject.Hospital.Exception.InvalidPatientException;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;



@GetMapping("/patients/age-range")
public ResponseEntity<Object> getPatientsByAgeRange(
        @RequestParam("minAge") int minAge,
        @RequestParam("maxAge") int maxAge) {
    try {
        return ResponseEntity.ok(service.getPatientsByAgeRange(minAge, maxAge));
    } catch (InvalidPatientException exception) {
        return ResponseEntity.badRequest()
                .body(new ApiError(exception.getMessage()));
    }
}
```

Age-range checks

```
GET /hospital/patients/age-range?minAge=18&maxAge=40
→ includes ages 18 and 40; excludes ages 17 and 41

GET /hospital/patients/age-range?minAge=18&maxAge=18
→ only patients aged exactly 18

GET /hospital/patients/age-range?minAge=0&maxAge=120
→ all valid-age patients

GET /hospital/patients/age-range?minAge=40&maxAge=18
→ 400, {"message":"minAge must not exceed maxAge"}

GET /hospital/patients/age-range?minAge=-1&maxAge=40
→ 400, {"message":"Ages must be between 0 and 120"}

A valid range with no patients → 200 []
Compare GET /hospital/patients before and after → stored data unchanged
```

11. Recap questions

Where is the doctor relationship persisted? Why does a valid empty search return 200 []? Which request proves a read-only calculation left H2 unchanged?

12. Doctor workload balancing

Problem: workload report and one balancing step

Extend the existing Hospital H2 application with a workload report and one balancing operation. Workload means the number of patients currently assigned to a doctor. Use the existing models, JdbcTemplate and named row mappers. Calculate counts and make decisions in Java; SQL joins and aggregation are not required.

Endpoint 1: GET /doctors/workload. Return doctorId, doctorName and patientCount for every doctor, including doctors with zero patients. Sort by patientCount descending, then doctorId ascending. No doctors means 200 with [].

```
[
  {"doctorId":1,"doctorName":"Dr Watson","patientCount":5},
  {"doctorId":2,"doctorName":"Dr Banner","patientCount":1},
  {"doctorId":3,"doctorName":"Dr Strange","patientCount":0}
]
```

Endpoint 2: POST /doctors/rebalance, with no request body. Choose the busiest and least busy doctors. If their counts differ by more than one, move exactly one patient from busiest to least busy. For tied doctor counts, choose the lowest doctor ID. Move the lowest-ID patient belonging to the busiest doctor. Persist the new assignment using the existing UPDATE method. Ignore unassigned patients.

Return 200 with moved, message, patientId, fromDoctorId and toDoctorId. With fewer than two doctors, or a maximum/minimum difference of at most one, return moved=false and a useful message; the three IDs are null. A request performs one move, not a complete redistribution.

```
Before: Watson 5, Banner 1, Strange 0
One POST → move lowest-ID Watson patient to Strange
After:  Watson 4, Banner 1, Strange 1
```

Use the existing Controller → Service → Repository flow. No new database tables, SQL joins or SQL aggregation are required. Response models are not database entities.

Predict the results for workloads 5/1/0, 2/1, all zero, and a single doctor. With counts doctor 1:2, doctor

2:2, doctor 3:0, doctor 4:0, which source and target should be selected?

Reference solution: response models

Model/DoctorWorkload.java

```
package com.accio.springProject.Hospital.Model;

public class DoctorWorkload {
    private final long doctorId;
    private final String doctorName;
    private final int patientCount;

    public DoctorWorkload(long doctorId, String doctorName, int patientCount) {
        this.doctorId = doctorId;
        this.doctorName = doctorName;
        this.patientCount = patientCount;
    }

    public long getDoctorId() { return doctorId; }
    public String getDoctorName() { return doctorName; }
    public int getPatientCount() { return patientCount; }
}
```

Model/RebalanceResult.java

```
package com.accio.springProject.Hospital.Model;

public class RebalanceResult {
    private final boolean moved;
    private final String message;
    private final Long patientId;
    private final Long fromDoctorId;
    private final Long toDoctorId;

    public RebalanceResult(boolean moved, String message,
            Long patientId, Long fromDoctorId, Long toDoctorId) {
        this.moved = moved;
        this.message = message;
        this.patientId = patientId;
        this.fromDoctorId = fromDoctorId;
        this.toDoctorId = toDoctorId;
    }

    public boolean isMoved() { return moved; }
    public String getMessage() { return message; }
    public Long getPatientId() { return patientId; }
    public Long getFromDoctorId() { return fromDoctorId; }
    public Long getToDoctorId() { return toDoctorId; }
}
```

Reference solution: repository

Add findAll to the existing DoctorRepository. Retain the named doctorMapper and existing patient methods. findPatientsByDoctorId already orders patients by ID.

```
public List<Doctor> findAll() {
    return jdbc.query(
            "SELECT id, name FROM doctors ORDER BY id", doctorMapper);
}
```

Reference solution: workload service and controller

Add these imports to DoctorService, retaining its existing imports and constructor:

```
import com.accio.springProject.Hospital.Model.DoctorWorkload;
import com.accio.springProject.Hospital.Model.RebalanceResult;
import java.util.ArrayList;
import java.util.Comparator;



public List<DoctorWorkload> getWorkload() {
    List<DoctorWorkload> result = new ArrayList<>();
    for (Doctor doctor : repository.findAll()) {
        int count = repository.findPatientsByDoctorId(doctor.getId()).size();
        result.add(new DoctorWorkload(doctor.getId(), doctor.getName(), count));
    }

    result.sort(Comparator.comparingInt(DoctorWorkload::getPatientCount)
            .reversed()
            .thenComparingLong(DoctorWorkload::getDoctorId));
    return result;
}
```

In DoctorController, import DoctorWorkload and java.util.List; retain the existing /doctors class mapping:

```
@GetMapping("/workload")
public ResponseEntity<List<DoctorWorkload>> getWorkload() {
    return ResponseEntity.ok(service.getWorkload());
}
```

Reference solution: balancing service and controller

Add the following method inside DoctorService. The report is sorted by descending workload and ascending doctor ID. The least-busy selection uses its own lowest-ID tie-break.

```
public RebalanceResult rebalance() {
    List<DoctorWorkload> workloads = getWorkload();
    if (workloads.size() < 2) {
        return new RebalanceResult(false, "At least two doctors are needed",
                null, null, null);
    }

    DoctorWorkload busiest = workloads.get(0);
    DoctorWorkload leastBusy = workloads.get(0);
    for (DoctorWorkload current : workloads) {
        if (current.getPatientCount() < leastBusy.getPatientCount()
                || (current.getPatientCount() == leastBusy.getPatientCount()
                && current.getDoctorId() < leastBusy.getDoctorId())) {
            leastBusy = current;
        }
    }

    if (busiest.getPatientCount() - leastBusy.getPatientCount() <= 1) {
        return new RebalanceResult(false, "No balancing needed",
                null, null, null);
    }

    List<Patient> patients =
            repository.findPatientsByDoctorId(busiest.getDoctorId());
    if (patients.isEmpty()) {
        return new RebalanceResult(false, "Workload changed; try again",
                null, null, null);
    }

    long patientId = patients.get(0).getId();
    boolean updated = repository.assignPatient(leastBusy.getDoctorId(), patientId);
    if (!updated) {
        return new RebalanceResult(false, "Patient no longer exists; try again",
                null, null, null);
    }

    return new RebalanceResult(true, "One patient reassigned",
```

```
            patientId, busiest.getDoctorId(), leastBusy.getDoctorId());
}
```

In DoctorController, import RebalanceResult and add:

```
@PostMapping("/rebalance")
public ResponseEntity<RebalanceResult> rebalance() {
    return ResponseEntity.ok(service.rebalance());
}
```

Verification

GET workload must be read-only. A successful POST moves exactly one patient and preserves the total assigned count. Unassigned patients remain unchanged. Check source and target patient lists, actual moved IDs, zero-patient doctors, both tie-breaks and no-move cases. Restart using the same H2 file to confirm persistence.

This exercise evaluates sequential requests. Its read/choose/update sequence does not provide protection against simultaneous balancing requests.